

COS 214 Practical 3

Code Masters

Team Members:

Kyle Pillay - 23545756

Takunda Mugwagwa - 24667341

Brett Pitts – u24682251

Task 1: Event concept and completed architecture (45 marks)

Your team must first decide what kind of event EventFlow is coordinating. The event must be large enough to justify nested structure and changing notifications.

- 1.1 Define your event in approximately one page or less. Give it a name, explain what happens there, and identify at least **five concrete kinds of event units** and at least **three kinds of grouping/area** that could appear in the Composite tree.

1.1 PixelCon is a multi-day gaming convention held across a single indoor/outdoor venue. Attendees move between gaming halls, a competitive tournament zone, and an outdoor plaza containing food, merchandise, and ticketing. The event runs on a live coordination system: a central control point can issue notices, schedule changes, capacity warnings, weather alerts, and evacuation instructions, that must reach the correct operational units without any single class needing to know about every other class in the venue.

The venue is large enough to justify nested structure. The root event contains multiple halls: one hall contains a further nested zone dedicated to tournament play; and every hall contains several individually reactive operational units. This gives PixelCon three levels of Composite nesting below the root, and units that behave meaningfully differently from one another even though they all respond to the same shared interface.

Groupings / areas

1. PC Hall — houses PC gaming stations and VR demo booths, plus a nested sub-zone.
2. Tournament Zone — nested *inside* PC Hall; hosts the competitive bracket play for the event.
3. Console Hall — houses console and handheld gaming stations, plus an information desk.
4. Outdoor Plaza — houses mobile gaming, food, merchandise, and ticketing — the only area directly exposed to weather.

Concrete event units

1. PC Station — a desktop gaming seat. Indoor unit; ignores weather alerts entirely, but shuts down immediately on evacuation.
2. Console Station — a console gaming seat. On close, finishes and saves any active session rather than dropping it instantly.
3. Switch Station — a portable handheld station. Runs in a reduced-capacity "portable mode" and otherwise ignores most notices except evacuation.
4. VR Booth — a VR demo pod. Enforces a cooldown period before it can reopen after closing, and pauses new sessions during a capacity alert.
5. Tournament Desk — manages bracket registration and round scheduling, reacts to schedule-change notices by delaying the active round.
6. Info Desk — never fully closes, only reduces staffing; updates its announcement board in response to every notice type it receives, unlike most units which only react to specific ones.
7. Merch Vendor — flags itself for restock during a capacity alert.
8. Food Vendor — closes on a weather alert, and finishes a mid-order before shutting down.
9. Mobile Station — a lightweight mobile-gaming station with near-instant open/close and a queue that reshuffles on schedule changes.
10. Ticket Booth — the only unit that stays locked shut after an evacuation until an explicit resume notice arrives, rather than reopening freely.

- 1.2 Identify the GoF participants of both Observer and Composite in your design. If one class participates in more than one role, list each role separately and explain the collaboration in which that role applies.

1.2

Main plays the clients for all.

Composite pattern participants

- Component = EventComponent
- Composite = EventGroup
- Leaf = EventUnit and its ten concrete descendants: PCStation, ConsoleStation, SwitchStation, MobileStation, VRBooth, TournamentDesk, InfoDesk, MerchVendor, FoodVendor, TicketBooth

- Client = main.cpp

Observer pattern participants

- Subject = Subject (abstract)
- ConcreteSubject = EventGroup, EventControl
- Observer = Observer (abstract)
- ConcreteObserver = EventGroup, and the ten concrete EventUnit descendants listed above

EventGroup participates in three roles:

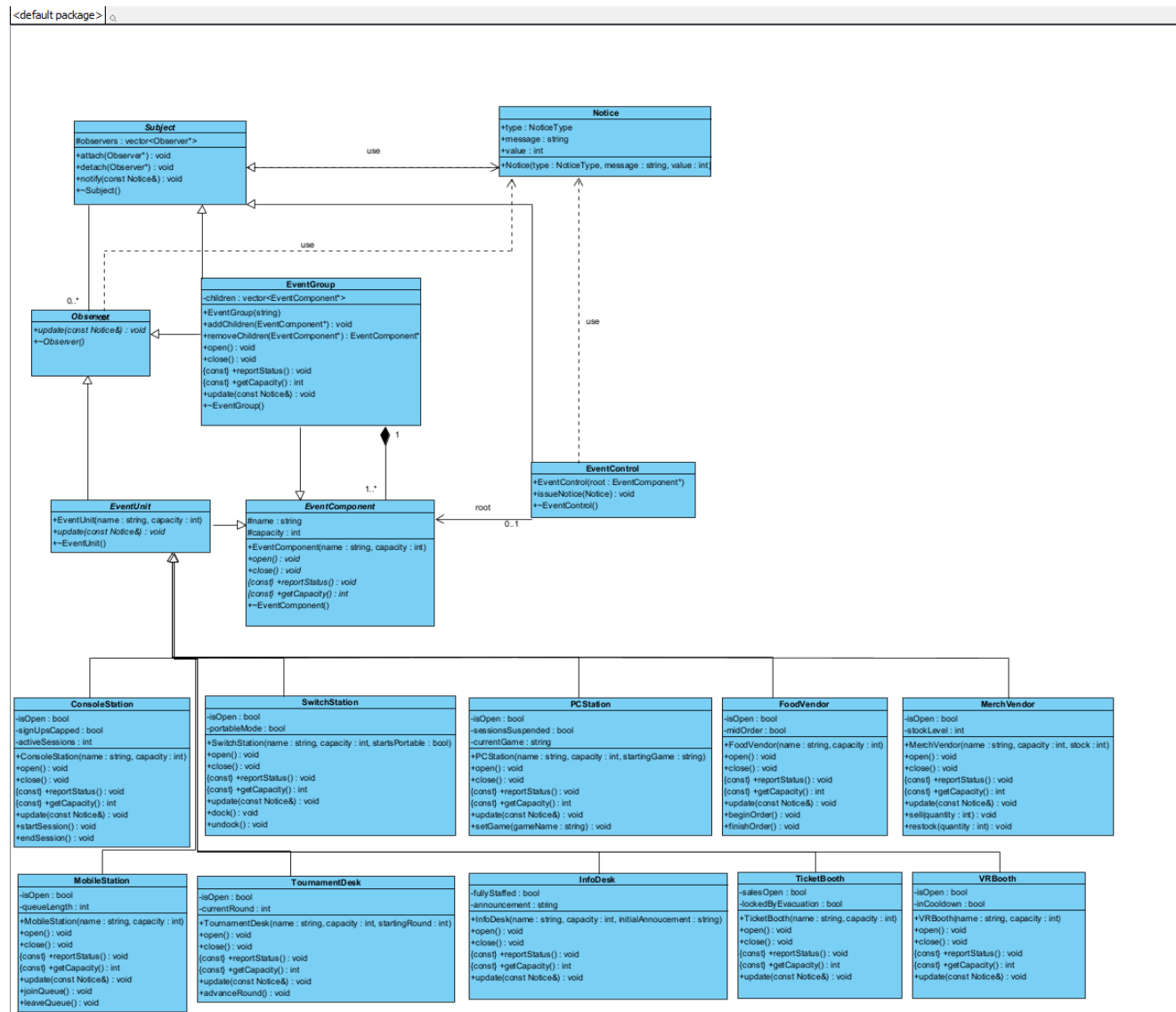
- As Composite, it owns and recurses over its children.
- As ConcreteSubject, it can have observers attached to it (typically, but not necessarily, its own children) and pushes notices to them.
- As ConcreteObserver, it can itself be registered with a parent EventGroup (or with EventControl), so a notice issued above it is received via update() and then re-broadcast downward via its own notify(). This is what makes a notice cascade through multiple tree levels.

EventUnit (and every concrete leaf), participates in two roles:

- As Leaf, it is a terminal node in the Composite.
- As ConcreteObserver, it can be attach()ed to whichever EventGroup contains it and reacts to notices via update().

1.3 Complete the UML class diagram for your full EventFlow design. Show abstract classes/operations, inheritance, associations, multiplicities, ownership, observer registrations, relevant attributes and operations, and a virtual destructor on every polymorphic base. Your diagram must include at least five concrete leaf types and enough structure to support every scenario in Task 5.

1.3



- 1.4 Write a design rationale. Explain: (a) why your Composite is a genuine part-whole tree; (b) why Observer is required rather than direct hard-coded calls; (c) who owns event components; (d) who owns or does not own observer references; and (e) why any class that is both Subject and Observer is not a misuse of either pattern.

1.4

a) Composite as a genuine part-whole tree. EventGroup doesn't just reference its children, it owns them: each child is added via `addChildren()`, belongs to exactly one group, and is deleted when that group's destructor runs. Since a child can itself be an EventGroup, deletion recurses, `delete pixelCon` releases the entire subtree. That ownership contract is what makes it a tree, not a plain list.

(b) Why Observer is required. Without it, EventControl would need to hard-code a call to every concrete unit for every notice, rewritten each time a unit moves or changes. `attach/detach/notify` let a subject broadcast to whoever is currently registered without knowing their concrete type.

(c) Who owns event components. EventGroup owns its children exclusively, `addChildren()` takes ownership, `~EventGroup()` deletes each child once, and `removeChildren()` hands ownership back without deleting, so a unit can be safely re-parented. The root is owned by whoever creates it in `main.cpp`.

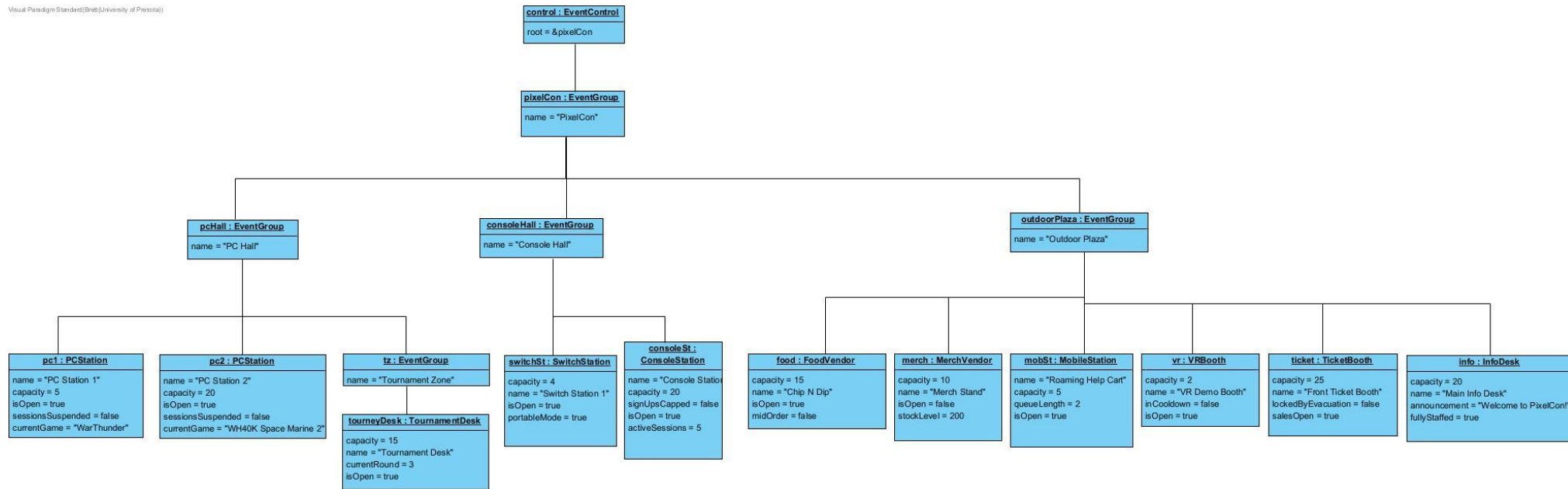
(d) Who owns observer references. Subject never owns its observers, `observers` is a vector of raw, non-owning pointers, and `~Subject()` just clears the list. lifetime is decided by the Composite tree, registration is a freely changeable relationship. If Subject owned observers too, moving a unit between groups would risk a double-free.

(e) Why EventGroup being Subject and Observer isn't a misuse. Ownership lives in children; notification lives in observers; being notified is a third link entirely, tracked by nothing more than being attached somewhere. `addChildren()` never `auto-attach()`s, so a child can be structurally contained without listening for notices, proving the two collaborations stay genuinely separate even when the same object plays both roles.

2.3 Build a sample event with at least **three levels of nesting**. Draw an **object diagram** showing the concrete object instances and the ownership tree.

2.3

Visual Paradigm Standard (Brett, University of Pisa)



2.4 Demonstrate destruction of the root and explain why the complete owned subtree is released exactly once. If an object can be transferred between composites, define the ownership rules for that transfer clearly.

2.4

```
delete pixelCon;
```

pixelCon owns its child `EventComponent*` pointers inside its children vector. Its destructor recursively loops through and deletes each child element, ensuring every object allocated with `new` in `main.cpp` is released exactly once — because each child belongs to exactly one group at a time, `delete pixelCon` triggers a cascade through `pcHall`, `consoleHall`, and `outdoorPlaza`, and each of those destructors deletes their own children in turn, all the way down to `tourneyDesk`. No object is ever owned by two groups at once, so nothing gets deleted twice and nothing leaks.

If an object is transferred between composites, the ownership rule is: only one `EventGroup` may own it at any moment. `removeChildren(child)` hands ownership back to the caller without deleting it (and also detaches the child as an observer if it was registered), and the caller must immediately re-parent it via another group's `addChildren(child)` before that pointer is dropped. This is exactly what happens in `main.cpp` when `mobileStation` is relocated from `outdoorPlaza` to `consoleHall`: `outdoorPlaza->removeChildren(mobileStation)` releases ownership, then `consoleHall->addChildren(relocated)` re-establishes it, so the object is never owned by two groups at once and never left owned by none.

3.1 Implement the Subject abstraction and its registration list. `attach` and `detach` must behave sensibly for duplicate registration and attempts to detach an observer that is not registered. State your policy.

3.1 `attach(observer)` first rejects `NULL`, then linearly scans observers for the pointer already being present; if it's found, `attach()` is a no-op and the observer is not added twice — this prevents `notify()` from ever calling `update()` on the same observer more than once per notice. If the pointer isn't already registered, it's pushed onto the back of the vector.

`detach(observer)` scans for a matching pointer and erases it if found; if the observer was never registered (or was already detached), `detach()` simply does nothing and returns — no error, no crash, just a safe no-op either way.

Both methods treat observers purely as a set of raw, non-owning identities: `Subject` never owns or deletes what these pointers point to (see `~Subject()`, which only calls `observers.clear()`), so `attach/detach` are just membership toggles on that list, not lifetime operations. This keeps the policy simple and predictable: calling `attach()` twice

is harmless, calling detach() on something not present is harmless, and neither can leave the list in an inconsistent state.

3.2 Implement the Observer abstraction and the required concrete observing behaviour. Destruction must not leave dangling registrations. Explain whether observers detach themselves, subjects clean registrations, or your design guarantees lifetime ordering in another defensible way.

3.2)

Observer implementation and lifetime safety. Observer is a pure abstract base with one pure virtual method, update(const Notice&), and a virtual destructor that does nothing itself (Observer::~~Observer(){}), it holds no state, so there's nothing for it to clean up on its own. EventUnit inherits from both EventComponent and Observer, giving every concrete leaf a dual role: Leaf in the Composite tree, and ConcreteObserver reacting to notices. EventUnit::update() stays pure virtual, forcing every concrete leaf (TicketBooth, VRBooth, etc.) to define its own reaction, and EventUnit's own destructor is empty since it owns no resources itself, each concrete leaf's destructor handles anything it individually added.

Our design does not rely on observers detaching themselves in their own destructor. Instead, it guarantees lifetime ordering structurally, through ownership: every EventUnit is owned by exactly one EventGroup, and that same EventGroup is responsible for cleaning up its registration before it's ever destroyed.

3.4 Make at least one Composite-level class capable of receiving a notification from above and then notifying interested observers below. Demonstrate a notification cascading through at least three runtime levels.

3.4

```
control.issueNotice(Notice(NoticeType::SCHEDULE_CHANGE,  
                           "Bracket matches pushed back 30 minutes", 1));
```

Causes:

```
[PixelCon] received notice: Bracket matches pushed back 30 minutes  
[PC Hall] received notice: Bracket matches pushed back 30 minutes  
[Tournament Zone] received notice: Bracket matches pushed back 30 minutes  
Tournament Desk - next match delayed, desk closed for rescheduling.  
[Console Hall] received notice: Bracket matches pushed back 30 minutes  
Main Info Desk updated the announcement board.  
[Outdoor Plaza] received notice: Bracket matches pushed back 30 minutes  
Roaming Help Cart reshuffled its queue, redirecting 1 attendees elsewhere.
```

3.5 Choose push or pull for your Observer implementation. Explain the trade-off and show the exact state/message information transferred in your design.

3.5 We use push as notify() passes a full Notice straight into update(const Notice&) observers never call back for more data.

```
struct Notice {  
    NoticeType type;  
    std::string message;  
    int value;  
};
```

Trade-off. no stored current state, no back-reference needed from observer to subject and we don't have to call many functions to figure out what the notice is about. The cost is rigidity ,every notice type has to fit the same Notice shape, even for leaves. Pull would scale better if observers needed very different data but would require each observer to know its subject's interface. For EventFlow's small, uniform notices, Push is the simpler fit.

Task 4: Event rules and dynamic behaviour (25 marks)

Your event should feel like a system rather than a demonstration of isolated pattern classes.

4.1 Define at least **five event rules** that cause concrete units to react differently to the same notice. Example: during a weather alert an outdoor stage pauses, an indoor hall remains open, a shuttle changes route and a medical unit remains active. Achieve these differences through polymorphism and object state, not type checking in a controller.

4.1

Rule 1: WEATHER_ALERT distinguishes indoor from outdoor units.

FoodVendor::update() calls close() on WEATHER_ALERT because it's an outdoor unit, with an explicit comment marking it as such. PCStation::update() has a WEATHER_ALERT case that does nothing (comment: // indoor unit), and ConsoleStation/SwitchStation/MerchVendor don't even list WEATHER_ALERT as a case,

falling to default: break. MobileStation reacts a third way — clearing its queue but staying conceptually open, since it can physically relocate rather than shutting down. Same notice, three distinct polymorphic responses driven by what kind of unit each object is.

Rule 2: CAPACITY_ALERT produces suspension in some units, cosmetic logging in others, and silence elsewhere.

TicketBooth stops selling (salesOpen = false), PCStation and VRBooth suspend/cooldown their sessions, ConsoleStation caps sign-ups (halving effective capacity), while MerchVendor only logs a "flagged for restock" message without changing any state, and SwitchStation/FoodVendor/TournamentDesk ignore it entirely.

Rule 3: EVACUATE triggers graceful shutdown with different "cleanup" semantics per unit.

ConsoleStation::close() saves active sessions before shutting down.

FoodVendor::close() finishes the current order first. TicketBooth doesn't just close — it sets lockedByEvacuation = true, meaning it will refuse to reopen even if open() is called again, until a RESUME notice clears the lock. VRBooth force-ends its session and enters cooldown. Same trigger, four different shutdown behaviors, each expressed through the object's own state and close() override rather than a shared branch.

Rule 4: RESUME doesn't just "undo" — it's asymmetric with what caused the suspension.

TicketBooth::update() on RESUME only clears lockedByEvacuation — it does not automatically reopen sales (salesOpen stays false until someone explicitly calls open()), reflecting that "cleared to resume" and "actually open" are different states. PCStation and VRBooth clear their suspension flags directly. TournamentDesk goes further and reopens immediately, printing which round resumes. ConsoleStation uncaps sign-ups. This shows RESUME isn't a single uniform "reverse everything" branch — each unit decides what resuming actually means for it.

Rule 5: SCHEDULE_CHANGE affects queue-based units differently from fixed-session units.

MobileStation::update() redirects part of its queue (min(queueLength, notice.value)), using the notice's numeric value field meaningfully. TournamentDesk::update() closes

the desk entirely for rescheduling. InfoDesk just updates its announcement board text. Units like SwitchStation, FoodVendor, and MerchVendor don't react at all (no case for it) — showing the same notice type produces a spectrum from "hard close" to "text-only update" to "no reaction," entirely through each leaf's own `update()` override.

All five are achieved purely through each concrete leaf's own state and its override of `update()` — there's no central `if (typeid(x) == ...)` dispatch anywhere; `EventGroup::update()` just cascades the notice via `notify()`, and each leaf decides its own reaction.

4.4 Add at least **three original features** that are not explicitly required above. They must fit your event and should create interesting interactions worth modelling. Briefly explain why each feature could be added without turning one class into a god object.

4.4

Feature 1: Lockout states that outlast their triggering notice.

TicketBooth's `lockedByEvacuation` flag and VRBooth's `inCooldown` flag both create a state that persists independently of the immediate notice — a locked ticket booth refuses `open()` calls entirely until explicitly cleared by `RESUME`, and a cooling-down VR booth refuses new sessions until `RESUME` arrives too. This adds a believable "can't immediately snap back to normal" behavior without any single class growing new responsibilities — each leaf just tracks one private `bool` and checks it in `open()`.

Feature 2: Stock-based capacity semantics (MerchVendor).

Rather than every leaf's `getCapacity()` meaning "seats/headcount," `MerchVendor::getCapacity()` deliberately repurposes the same virtual interface to mean "remaining stock," with `sell()/restock()` operations that never let stock go negative. This shows the common `EventComponent` interface can carry different real-world meaning per leaf while still aggregating sensibly through `EventGroup::getCapacity()`'s summation — no changes needed anywhere else in the tree.

Feature 3: Order-in-progress tracking (FoodVendor).

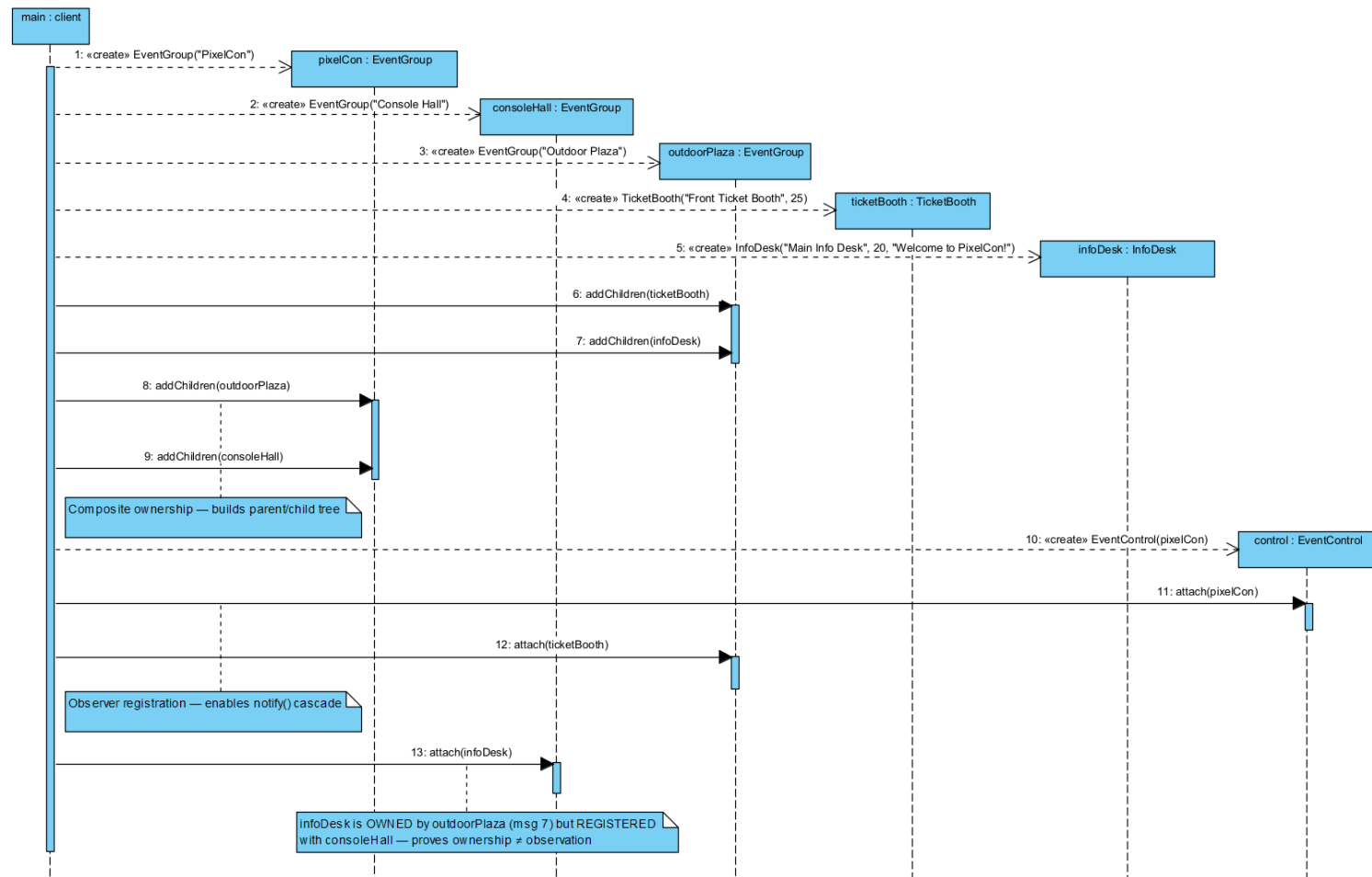
`FoodVendor::beginOrder()/finishOrder()` track whether a customer transaction is mid-flight, and `close()` checks this before shutting down, printing a "finishing current order before closing" message rather than cutting off service instantly. This is a small but genuine behavioral difference from `ConsoleStation`'s similar "save active sessions"

idea, showing the same concept (graceful shutdown) implemented as a different mechanic per leaf, which avoids the two classes converging into copy-pasted logic.

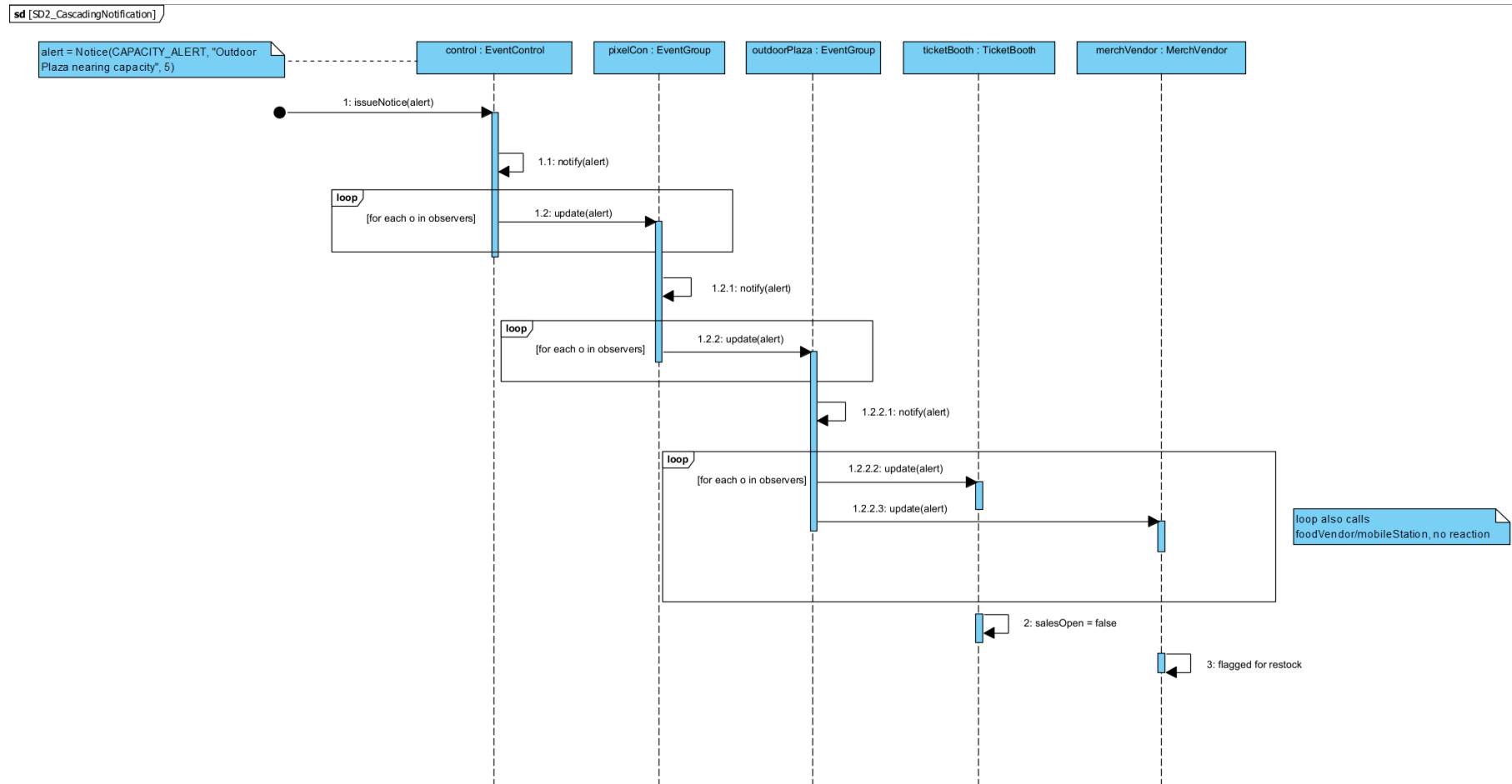
Each of these stays contained to a single leaf's private state and its own overridden methods — none of them require `EventGroup`, `EventControl`, or `main.cpp` to know about the specific leaf type, so no class becomes a god object coordinating behavior on others' behalf.

5.1 SD1 – Building and registering part of the event. Show the client creating a root Composite, at least one nested Composite and at least two Leaves, adding them to the ownership tree, and establishing the Observer registrations needed for later notifications. Creation messages must begin lifelines at the correct time. The interaction must make it possible for a marker to distinguish *Composite ownership* from *Observer registration*.

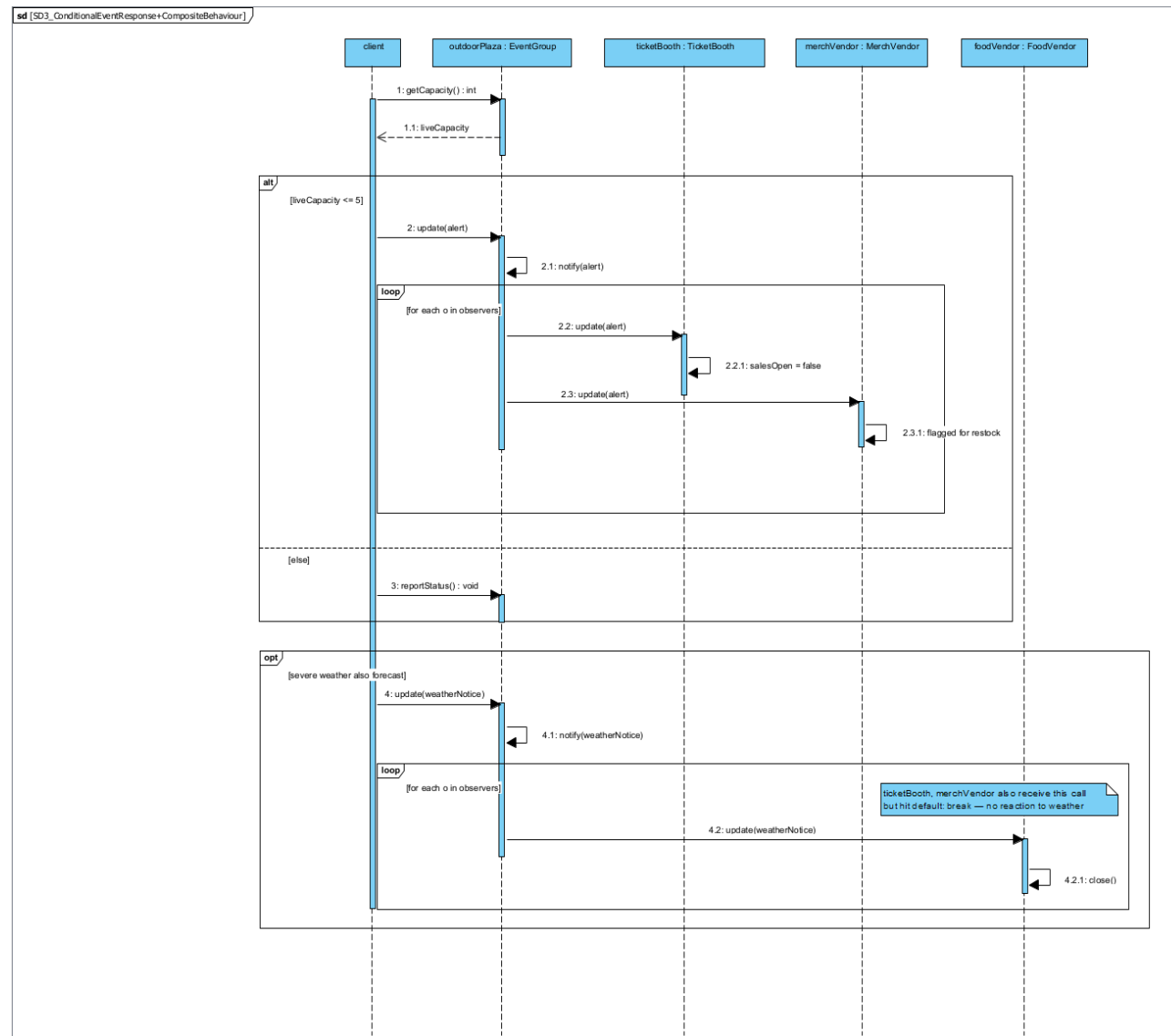
sd [SD1_BuildingAndRegistering]



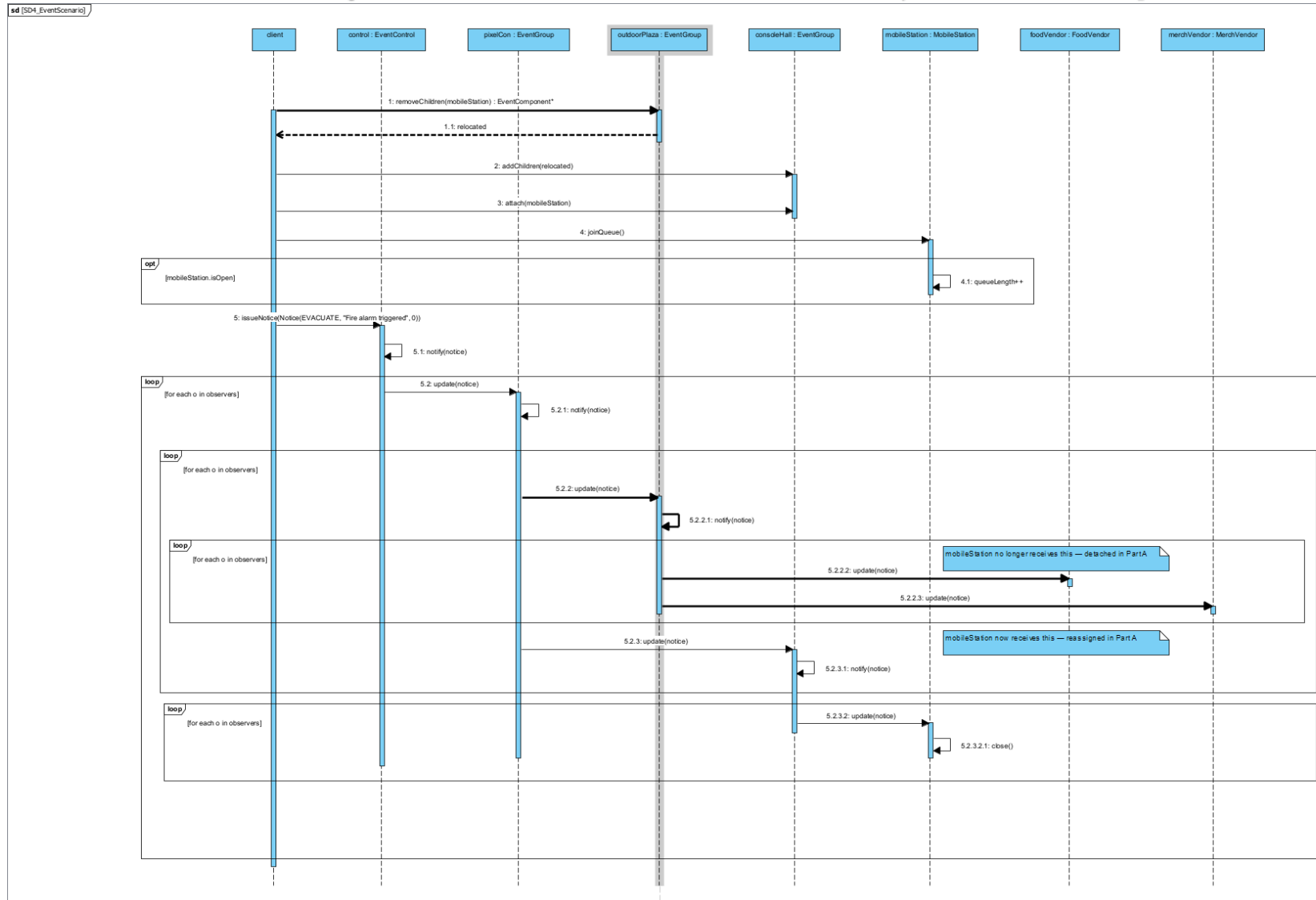
5.2 SD2 – Cascading event notification. Show a realistic notice originating from an appropriate Subject and cascading through at least three runtime levels to multiple observers. Use a `loop` fragment where observers are notified. If you chose pull, show the observer querying the subject; if you chose push, show the information passed to `update(...)`. At least two concrete event units must react meaningfully to the notice.



5.3 **SD3 – Conditional event response and Composite behaviour.** Model a realistic situation such as a capacity threshold, weather warning, schedule disruption, temporary closure or service interruption. The interaction must include a meaningful Composite operation and use an **alt** fragment with guards. Include an **opt** fragment where it naturally fits. Show different concrete objects responding through polymorphism rather than explicit type checking.



5.4 **SD4 – Your signature event scenario.** Invent a substantial interaction that makes your team’s event recognisably different from another team’s system. It must involve at least six lifelines, at least two Composite levels, at least one Observer notification, and at least one change during execution, such as an observer joining/leaving, a unit being reassigned, a pause followed by resume, a safety response followed by recovery, or end-of-event shut-down. If ownership or registration changes, show the relevant detach/attach or transfer messages clearly. Use at least one combined fragment and ensure the scenario can be traced to your actual C++ implementation.



Task 7: Git and GitHub engineering workflow(25 marks)

GitHub is part of the practical, not merely the place where the final code happens to live. You have only recently been introduced to branching workflows, so the emphasis in this practical is on **using GitHub consistently as a team** rather than mastering an advanced workflow.

7.1 Create one private GitHub repository for the team. The `README.md` must contain the event concept, team members, instructions for building with `make` and running the executable, Doxygen instructions and a brief architecture overview.

[Here](#)

7.4 In your submitted PDF, include a short GitHub workflow reflection (approximately half a page). Explain how the three members divided the work, how changes were integrated, and identify two or three commits or other repository activities that show meaningful collaboration. If your team used branches or pull requests, briefly explain how they helped.

Kyle completed:

1. Made Main UML diagram
2. Completed all sequence diagrams
3. Completed question 4.1, 4.4,
4. Fixed 2.4
5. Answered 3.1, 3.2 theory questions

Takunda completed:

1. Coded 5 leaves
2. Completed doxygen comments
3. Completed main.cpp

Brett completed:

1. Updated UML for 10 leaves
2. Completed Object diagram (2.3)
3. Coded all but lower 5 leaves(event units)
4. Completed Questions 1.4, 2.1, 2.2, 2.3, 2.4, 3.1, 3.2, 3.3, 3.4, 3.5, some of Question 6
5. Setup PDF

How work was done:

The team coordinated its work through branching and pull requests before merging to main. We used WhatsApp as a primary source of communication and reviewed each other's work where necessary before committing and adding on or fixing each other's work where possible. If improvements could be made, communication was made and a solution was reached before any major changes were committed to ensure reliability and reduce possible errors. The team coordinated its work through branching and pull requests before merging to main. We used WhatsApp as a primary source of communication and reviewed each other's work where necessary before committing or adding on or fixing each other's work where possible. If improvements could be made, communication was made and a solution was reached before any major changes were committed to ensure reliability and reduce possible errors.

Three commits that show this in practice: Brett's "Merge pull request #7 from KP23545756/feature/first5leaves+makefile" followed shortly by "Fixed errors and makefile" shows a branch being reviewed, merged, and then repaired once integration issues surfaced like the review-before-major-changes process described above. Kyle's "Fixed naming inconsistency in UML Diagram" commit reflects the class diagram being corrected after a review caught a mismatch with the implemented addChildren/removeChildren methods, rather than being drawn once and left untouched as well as committing the sequence diagrams. Takunda's steady run of incremental commits ("info desk is officially done", "doing last 5 leaves", "ticket booth implementation done") over multiple sessions shows ongoing development across the practical rather than a single end-of-project upload.

Evidence:

Kyle

All Sequence Diagrams complete ● KP23545756 authored 17 minutes ago	Verified 94aedb5	📄 <>
Fixed naming inconsistency in UML Diagram ● KP23545756 authored 18 minutes ago	Verified dba3d1d	📄 <>
Starter Document 📄 ● KP23545756 authored 3 days ago	Verified c444d68	📄 <>
Sequence Diagram 4 for Task 5 ● KP23545756 authored 1 minute ago	Verified f655d09	📄 <>
Sequence Diagram 3 for Task 5 ● KP23545756 authored 1 minute ago	Verified 0f22ed7	📄 <>
Sequence Diagram 2 for Task 5 ● KP23545756 authored 1 minute ago	Verified c1a3b37	📄 <>
Sequence Diagram 1 for Task 5 ● KP23545756 authored 2 minutes ago	Verified 807a9c0	📄 <>

Brett

Commits on Aug 31, 2025		
Updated doxygen comments	95c9003	🔗
Finished Object Diagram, added parts of word pdf	ee40562	🔗
Fixed Broken Code and made it match description	e433b66	🔗
Merge branch 'main' of https://github.com/KP23545756/Practical_3	0108a55	🔗
Update README.md	7861678	🔗
Merge pull request #7 from KP23545756/feature/FirstLeaves+makefile	Verified c5b2a6c	🔗
Add valgrind rule to makefile	c19806a	🔗
Fixed errors and makefile	e4b405e	🔗
Merge pull request #5 from KP23545756/feature/FirstLeaves+makefile	Verified e918a5a	🔗
Update README.md	4138f78	🔗
Added makefile	49f77eb	🔗
Finished First 5 Leaves, need debugging tho	5b2a399	🔗
Finished .h files	2285a06	🔗
Setup for First 5 leaves	d259467	🔗
Commits on Aug 30, 2025		
Updated PDF	35a510b	🔗
Updated and uploaded UML	8ac9a3a	🔗
Create COS 214 Practical 3 PDF.docx	e718511	🔗
Update README.md	e5ae169	🔗
Update README.md	4b48386	🔗
Merge branch 'main' of https://github.com/KP23545756/Practical_3	7aef89ea	🔗
Update readme for task 7.1	9374802	🔗
Merge pull request #4 from KP23545756/feature/Event-control	Verified 52776cb	🔗
Implemented Event group and event Control	2029a08	🔗
Merge pull request #3 from KP23545756/feature/event-component	Verified f6ab15a	🔗
Merge pull request #2 from KP23545756/feature/Observer++-subject	Verified ebf71e2	🔗
Implemented Event group	61a5760	🔗

Your branches

Branch	Updated
feature/FirstLeaves+makefile	🔗 13 hours ago
feature/Event-control	🔗 2 days ago
feature/event-component	🔗 2 days ago
feature/Observer++-subject	🔗 2 days ago

Active branches

Branch	Updated
feature/FirstLeaves+makefile	🔗 13 hours ago
feature/Event-control	🔗 2 days ago
feature/event-component	🔗 2 days ago
feature/Observer++-subject	🔗 2 days ago

Takunda

Main.cpp done. you can edit anywhere you see fit
last Implementation
ticket booth implementation done
last .h file implementation
implementing last 5 leaves
info desk is officially done
infoDesk.h file is done
doing last 5 leaves!
doing last 5 leaves. TournamentDesk done
Add files via upload
Add files via upload

Commits on Sep 1, 2026

doxygen comments added topboy-taku authored 44 minutes ago	Verified ddb90f0	 <>
doxygen comments added topboy-taku authored 46 minutes ago	Verified 691e9fd	 <>
doxygen comments added topboy-taku authored 47 minutes ago	Verified ea3fce3	 <>
doxygen comments added topboy-taku authored 48 minutes ago	Verified aad787a	 <>

Doxygen Proof:

My Project

[Main Page](#) [Related Pages](#)

Search

PixelCon

Repository for the Practical 3 files.

Team Members

- Kyle Pillay
- Takunda Mugwagwa
- Brett Pitts

Event Concept

PixelCon is a gaming convention coordinated by EventFlow. The venue is made up of a root event containing three halls; PC Hall, Console Hall, and Outdoor Plaza, with PC Hall further containing a nested Tournament Zone. Each hall contains individually reactive operational units (gaming stations, booths, desks, vendors) that respond differently to the same event-wide notices (schedule changes, capacity alerts, weather alerts, evacuation, etc.) depending on their own type and state.

Building

```
make  
./eventflow
```

Requires a C++11-compatible compiler.

Generating Documentation

```
doxygen Doxyfile
```

Open docs/html/index.html in a browser.

Architecture Overview

EventFlow combines Composite and Observer. EventComponent is the shared interface for EventUnit (Leaf) and EventGroup (Composite), letting a client treat a single station or an entire hall the same way. EventGroup also acts as both Subject and Observer, so notices issued by EventControl cascade down through nested groups to whichever concrete units are registered to receive them.

Project Structure